RQ

How can we make DP adaptive and resource-aware for TinyML-class federated learning, where devices have limited memory, energy, floating-point support, and communication capacity?

(1) Research the current state of Federated Learning (FL) on Microcontroller Units (MCUs) and low-resource IoT devices to identify primary hardware constraints such as RAM, Flash memory, and energy capacity.

The "SRAM/Flash Knee" remains the primary bottleneck for on-device training.

Primary hardware constraints for TinyML-class devices (e.g., ARM Cortex-M4, ESP32) include:

- RAM/SRAM: Typically ≤256 kB.
- Flash Memory: Generally 1–2 MB.
- Energy Capacity: < 100mJ per communication round. Encryption and noise generation can consume up to **20-40%** of the total per-round energy.
  Operations must remain in the milliwatt (mW) or microwatt (μW) range for battery or energy-harvesting longevity.
- Processing Speed: 1–400 MHz without the support of a conventional OS or DRAM
- A single FL round on an ESP32 (training + communication) consumes roughly 50–200 mJ; adding PPM overhead can multiply this 2–100× depending on the method.

MCU Architecture	RAM	Flash	Typical Energy
Arduino Uno (8-bit ATmega328)	2 KB	32 KB	~0.5-2 mJ/op
ARM Cortex-M4 (32-bit)	64-256 KB	256 KB-1 MB	~1-5 mJ/op
ESP32 (32-bit)	320-512 KB	4-16 MB	~2-8 mJ/op
ARM Cortex-M33 with TrustZone-M	128-512 KB	512 KB-2 MB	~1-6 mJ/op

(Our SLR and comprehensive survey on TinyML)

Solution: **Split federated Learning -> sends activation to server, server computes gradient.**
Li, J., & Kuang, R. (2022). Split Federated Learning on Micro-controllers: A Keyword Spotting Showcase. *ArXiv*. https://arxiv.org/abs/2210.01961

(2) Identify the unique challenges of implementing privacy-preserving methods (PPMs) in these settings, including lack of hardware acceleration for cryptography, limited floating-point precision for noise injection, and communication bandwidth bottlenecks.

Unique Implementation Challenges

- **Hardware Acceleration Gaps:** While the ESP32 and ARM Cortex-M33 provide hardware acceleration for **AES, SHA, and ECC**, they lack native support for the large-integer arithmetic required by **Paillier (Partial HE)** or **Lattice-based FHE**.
- **Floating-Point Precision:** Many MCUs use fixed-point arithmetic (INT8/INT16) to save power. Injecting Gaussian or Laplace noise for Differential Privacy (DP) often requires high-precision floats to maintain the **Privacy-Utility Trade-off**. Rounding errors may weaken formal DP calibration or reduce utility
- **Bandwidth Bottlenecks:** Low-power networks (LoRaWAN, NB-IoT) operate at <200 kbps. Cryptographic overhead (e.g., ciphertexts 10 times larger than plaintexts) can lead to **congestion collapse** or massive latency.
- **Heterogeneity of Noise Sensitivity:** Uniform privacy budgets across heterogeneous IoT nodes often lead to a decline in performance for minority classes, causing "fairness" issues in medical or industrial sensing tasks.

Challenge	Impact
No hardware crypto acceleration	Most MCUs lack AES/SHA accelerators, increasing encryption overhead 5-10×
Limited floating-point precision	Fixed-point arithmetic prevents Differential Privacy experts from accurately injecting noise.
Communication bandwidth bottlenecks	Low-power radios (LoRa, BLE) limit gradient upload frequency; multi-party protocols require many rounds
Energy capacity	Battery-powered devices cannot sustain intensive cryptographic operations

(3) Investigate specific research gaps and SOTA limitations in applying Differential Privacy (DP), Secure Aggregation, Homomorphic Encryption (HE), and Blockchain to MCU-based FL, focusing on the trade-off between privacy guarantees and resource consumption.

SOTA Limitations and Research Gaps

Differential Privacy (DP):

- Renyi DP & Local DP (LDP): Current SOTA uses Attribute-Adaptive Noise Injection, which scales noise based on the sensitivity of specific model layers.
- Scaling Limitation: On 8-bit MCUs, the lack of a true hardware Random Number Generator (RNG) makes generating cryptographically secure noise difficult. Pseudo-random generators (PRNGs) can be predictable, compromising the epsilon-guarantee.

For MCUs, two challenges arise:

- **tracking the Rényi divergence across rounds requires floating-point division and logarithm operations — feasible on FP32 MCUs (Cortex-M4F) but inaccurate on 8-bit fixed-point devices.(**RDP computation)
- **the Gaussian mechanism on FP32 hardware introduces rounding errors of ~10^{-7} in σ, which is negligible for weak privacy budgets (ε>5) but significant for strong privacy (ε<1) required in sensitive applications.(**Noise calibration)

Key gaps:

- No MCU-native DP-SGD library with fixed-point noise generation and formal ε/δ accounting.
- Per-sample gradient clipping — the cornerstone of DP-SGD — requires storing individual sample gradients: O(batch_size × model_size) RAM, which overwhelms a 512 KB device for any non-trivial model.
- Hybrid DP-FL work (Khalaf et al. 2024) shows 4–9% accuracy gains over standard FL, but targets server-class hardware.
- Privacy budget exhaustion: with limited local data on MCUs and many FL rounds, ε grows rapidly, offering weak privacy guarantees.

Hardware Solution:

DP-Box (hardware implementation) shows low overhead but requires custom hardware

Choi, W. S., Tomei, M., Vicarte, J. R. S., Hanumolu, P. K., & Kumar, R. (2018). Guaranteeing local differential privacy on ultra-low-power systems. In *Proceedings - 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture, ISCA 2018* (pp. 561-574). Article 8416855 (Proceedings - International Symposium on Computer Architecture). Institute of Electrical and Electronics Engineers Inc.. https://doi.org/10.1109/ISCA.2018.00053

Homomorphic Encryption (HE): Full HE provides high accuracy but is hindered by high computational costs and ciphertext expansion
Selective Homomorphic Encryption (SHE) attempts to mitigate this by only encrypting critical parameters, but it adds key management complexity

Secure Multi-Party Computation (SMPC): While more compute-efficient than HE, SMPC requires multiple interaction rounds, leading to high network overhead

Blockchain: It mitigates "single point of failure" risks but is described as "highly resource-intensive" for MCUs due to storage and energy drain

PPM	Primary Limitation on MCUs	Privacy-Resource Trade-off
Differential Privacy (DP)	Low-resolution sensors prevent quality noising; privacy budget exhausts quickly	High privacy requires large noise → significant accuracy degradation
Secure Aggregation	Requires $O(n^2)$ communication rounds for n parties; infeasible for large IoT networks	Strong privacy but high communication overhead
Homomorphic Encryption (HE)	FHE (BFV, CKKS) impractical on 8-bit MCUs; ciphertext 10-100×	PHE (Paillier) workable but $O(k^2)$ complexity dominates 63.6% of encryption time
Blockchain	Full consensus impossible on MCUs; ledger storage exceeds Flash capacity	Decentralized trust at cost of high energy/storage

(4) Evaluate the limitations of State-of-the-Art (SOTA) lightweight DP techniques (e.g., Renyi DP, local DP) and encryption schemes (e.g., Paillier, AES-based HE) when scaled down for 8-bit or 32-bit MCUs.

Qualitative and Quantitative Benchmarks

The following data represents typical performance for 32-bit MCUs (Cortex-M4/M33 or ESP32) during a single FL round.

Metric	Differential Privacy	Paillier HE (Partial)	Secure Aggregation (SMPC)	zk-SNARKs (ZKP)
CPU Cycles	Low (<10^5)	High (>10^8)	Moderate (10^6-10^7)	Very High (>10^9)
RAM Usage	5-10 KB (less than 10KB)	32-64 KB	16-32 KB	128-256KB
Flash Footprint	16 KB	50 KB	40 KB	150KB
Energy Impact	+5%	+35%	+15%	+50%
Comm. Overhead	0% (Noise only)	4X to 10X	1.5X to 2X	Fixed (~2000 bits)

Technique	Core Impact	Major SOTA Limitations	Resource Footprint (Reported)
Differential Privacy (DP)	Injects noise into model updates.	Accuracy degrades as privacy increases; privacy budget accumulation.	**Low CPU overhead**; 33% power reduction when combined with pruning. Villegas-Ch et al. (2024)
HE / SHE	Computes on encrypted data.	High ciphertext expansion and math complexity.	**High Memory/Energy**; complex for devices with ≤ 256 kB SRAM.
SMPC	Splits models into secret shares.	High communication costs due to multiple interaction rounds.	**Moderate CPU**; **High Bandwidth** requirements.
Blockchain	Decentralized, immutable logs.	Storage intensive; high energy drain for consensus.	14-18% reduction in decision latency but high storage/energy cost.
Model Pruning/ Quantization	Reduces model size.	Can lead to slower convergence or dropped accuracy.	**57% reduction** in data exchanged; 33% reduction in power

(5) Analyze the overhead of Blockchain and Secure Aggregation in low-resource IoT, specifically looking at consensus mechanism costs and multi-party communication requirements.

Blockchain: Costs are prohibitive for the MCU itself to act as a "Full Node." Instead, MCUs act as **Light Clients**, using **Simplified Payment Verification (SPV)**. The consensus mechanism (PoS or PoW) is offloaded to gateways; the MCU overhead is primarily the Flash cost of storing the block headers (100 KB).
Secure Aggregation: Requires multiple rounds of "handshaking." On an ESP32, this multi-party communication increases the active radio time by 200%-300% compared to standard FL.

(6) Evaluate the feasibility, gaps, and limitations of Secure Multiparty Computation (SMPC) protocols (e.g., secret sharing) for low-power devices.

Aspect	Finding
Energy consumption	SMPC and verifiable secret sharing consume **less energy** than HE nature
Computational intensity	Less computationally intensive than HE nature
Throughput	**Lower throughput** compared to other methods nature
Communication rounds	Fundamental limit on message rounds necessary for secure computation academia+1
IIoT optimization	Can minimize cryptographic operations but still requires communication overhead eccsubmit

SMPC is more feasible than HE for low-power devices but still requires significant multi-party communication.

(7) Research the role and limitations of Trusted Execution Environments (TEEs, such as ARM TrustZone-M) and hardware-based roots of trust in securing FL on MCUs.

Later.

(8) Explore emerging security methods like Zero-Knowledge Proofs (ZKPs) or lightweight anonymization techniques applicable to low-resource FL and their current research status.

Zero-Knowledge Proofs (ZKPs):’

Metric	Finding
Model accuracy	Maintains **almost identical accuracy** vs. standard FL
Time overhead	Execution time **on order of milliseconds**
Verification cost	Often **<50% of proof generation time**; can be offloaded to blockchain
Framework maturity	Systematic framework still **lacking**; Veri-CS-FL algorithm proposed for client selection
Current status	Emerging but not deployment-ready; future work needed on scalable/post-quantum ZKPs

(9) Gather quantitative benchmarks and performance data for CPU cycles, RAM usage (kilobytes), Flash usage, and energy consumption (milliJoules) for various PPMs running on common MCU architectures like ARM Cortex-M or ESP32.

Platform	Scheme	Encryption Time	Energy (per 100 ops)	Memory Footprint
Arduino Uno (8-bit)	FHE (BFV/CKKS)	**Impractical**	**Impractical**	Exceeds Flash
ESP32 (32-bit)	Paillier	Slow	Moderate	High
Raspberry Pi 4	BFV/CKKS	**<10 ms**	**<5 J**	Manageable

Operation / PPM	Platform	CPU Cycles (M)	RAM (KB)	Flash (KB)	Energy (mJ)	Round Time (s)
Baseline FL (no PPM)	ESP32 (240 MHz)	5–20 M	20–60 KB	32–64 KB	5–20 mJ	0.1–2 s
LDP (Laplace/Gaussian noise)	Cortex-M4	0.1–0.5 M overhead	+5–20 KB	+16–32 KB	+0.2–2 mJ	<0.1 s overhead
Central DP (DP-SGD clip+noise)	Cortex-M4	2–10× overhead	+10–50 KB	+32–64 KB	+0.5–5 mJ	0.5–5 s overhead
AES-128-CTR (per gradient block)	Cortex-M33 (CryptoCell)	0.05 M / KB	+2–8 KB	+8–16 KB	+0.05–0.5 mJ	Negligible
X25519 ECDH (SecAgg key gen)	Cortex-M4 (sw)	~8 M / pair	+16–32 KB	+48 KB	+1–5 mJ / pair	~0.05 s / pair
Paillier 2048-bit encrypt	Cortex-M4 (sw)	~200–500 M	+128–256 KB	+64 KB	+50–200 mJ	1.5–5 s
SecAgg full round (n=50)	Cortex-M4	~400–1000 M	64–256 KB	+48 KB	50–200 mJ	5–30 s
TrustZone-M world-switch	Cortex-M33	100–300 cycles / switch	+16–64 KB (secure)	+8–32 KB	+1–5 mJ/round	+15% overhead
Shamir 3-of-5 secret sharing	Cortex-M4	~10–50 M	+32–64 KB	+32 KB	+5–20 mJ	0.5–3 s

(10) Synthesize the collected data to compare how each privacy method affects model accuracy, training time, and device battery life.

Method	Accuracy effect	Training/round-time effect	Battery effect	Practical meaning
Local DP	Medium accuracy loss if noise high	Low time overhead	Low–medium energy	Best lightweight baseline
Rényi DP	Better privacy accounting than naive DP	Low–medium overhead	Low–medium	Good for formal privacy, but hard on fixed-point MCUs
Central DP / DP-SGD	Accuracy depends on clipping/noise	Medium–high overhead	Medium	Hard due to per-sample gradient memory
Secure Aggregation	No direct accuracy loss	High communication delay	High radio energy	Good privacy, but poor for unstable IoT
Paillier / PHE	No accuracy loss from encryption	Very high compute time	Very high energy	Usually too heavy for MCUs
SHE	Small/no accuracy loss if selected well	Medium–high overhead	Medium–high	Better than full HE, but partial privacy
Blockchain	No model accuracy effect	High synchronization delay	High storage/energy	Better for audit logs, not MCU-side consensus
SMPC	No direct accuracy loss	High communication rounds	Medium–high	Feasible only in small groups
TEE	No accuracy loss	Low–medium overhead	Low–medium	Good for key/seed protection
ZKP	No accuracy loss	High proof-generation cost	High	Promising for verification, not mature for MCUs
Split FL	May preserve accuracy	Medium communication dependency	Low–medium MCU cost	Strong candidate for TinyML

(11) Construct a comprehensive relationship table summarizing the PPMs, their SOTA limitations, and their specific resource footprints (CPU, Memory, Energy) in low-resource IoT environments.

PPM	SOTA Limitation on MCUs	CPU Overhead	RAM Usage	Flash Usage	Energy (mJ)	Best-Suited MCU
Local DP	Fixed-point prevents quality noising experts.	Low	<10 KB	<5 KB	1-3 mJ/op	Cortex-M4+
Rényi DP	Privacy budget exhausts quickly	Low-moderate	10-20 KB	5-10 KB	2-5 mJ/op	Cortex-M4+
Secure Aggregation	$O(n^2)$ communication rounds repository.	Moderate	20-50 KB	10-20 KB	5-15 mJ/op	ESP32+
Paillier (PHE)	$O(k^2)$ complexity; 63.6% of encryption time	Very high	50-100 KB	50-100 KB	10-30 mJ/op	ESP32+
AES-based HE	Ciphertext expansion 10-100×	High	30-80 KB	30-80 KB	8-25 mJ/op	ESP32+
BFV/CKKS (FHE)	**Impractical on 8-bit**; only feasible on 32/64-bit	Extreme	200-500 KB	200-500 KB	50-200 mJ/op	Raspberry Pi 4+
Blockchain (PoW/PoS)	Impossible on MCUs; requires lightweight consensus	Extreme	100-500 KB	>1 MB (ledger)	100-500 mJ/op	Not feasible

Blockchain (DPoS/PoR)	Still high overhead; throughput-latency tradeoff	High	50-200 KB	500 KB-1 MB	50-150 mJ/op	ESP32+
SMPC (Secret Sharing)	Low throughput; multi-party communication	Moderate	30-70 KB	20-50 KB	5-12 mJ/op	Cortex-M4+
TrustZone-M TEE	Requires Cortex-M33+; OP-TEE bottleneck	Low-moderate	50-150 KB (secure)	100-300 KB	3-10 mJ/op	Cortex-M33+
ZKPs (zk-SNARKs)	Framework maturity lacking; proof generation costly	High	50-150 KB	50-150 KB	10-25 mJ/op	ESP32+
Split FL (No PPM)	Requires server-side model; network dependency	Low	10-30 KB	20-50 KB	2-6 mJ/op	Arduino+

Approach for problem 1:

•Implement FL model using TFF

•Apply Differential Privacy (DP)

•Introduce adaptive noise based on:

•Data size

•Simulated device capability (low / medium / high)

■**Evaluation:**

•Accuracy vs Privacy vs Convergence

•Fixed noise vs Adaptive noise

Approach for problem 2

■Simulate different client types:

•Low resource → fewer epochs / skip rounds

•High resource → full participation

Define participation strategies:

•Full participation (baseline)

•Resource-aware participation (proposed)

■**Evaluation:**

•Training time

•Model accuracy

•Communication rounds

Approach for problem 3:

•Implement standard FL

•Add: Gradient clipping / sparsification (send fewer updates)

•Add DP noise (Starting with lightweight encryption)

Evaluation:

•Communication size

•Accuracy

•Privacy level (epsilon)

Overall Approach:

■Phase A

–*Build a **standard FL baseline + adaptive DP***

■Phase B

–***DP + selective client participation***

•*only some clients transmit in each round,*

■Phase C

–*DP + sparsification / clipping*

Baseline 0 -> Standard FL

Baseline 1 -> FL + fixed DP

Proposed Variant 1

-> FL + clipped/sparsified updates + DP

Proposed Variant 2

-> FL + DP + resource-aware participation

Proposed Variant 3

-> FL + DP + compression + resource-aware participation

DP Type	What it protects	MCU/TinyML relevance
Sample-level DP	One data sample from one client	Harder, needs per-sample clipping
Client-level DP	One entire client/device contribution	More natural for FL
Local DP	Client adds noise before sending update	More feasible for untrusted server
Central DP	Server adds noise after aggregation	Easier but assumes trusted server
RDP accounting	Tracks privacy loss over rounds	Useful, but accounting can be server-side

First -> **Client-side Local DP with server-side RDP accounting.**
Then DP SGD

Phase 1: Build baseline

Compare:

1. Standard FL
2. FL + fixed DP
3. FL + adaptive DP based on data size only
4. FL + adaptive DP based on data size + resource profile

Metrics:

- accuracy,
- convergence,
- ε/privacy budget,
- communication size,
- simulated energy cost,
- participation frequency.

Phase 2: Add resource-aware participation

Here, the client decision should depend on:

- battery level,
- memory profile,
- bandwidth condition,
- previous participation,
- remaining privacy budget,
- local data size.

clients participate only when the expected utility gain justifies the privacy and resource cost.

Example decision rule:

> A client participates in round t only if its resource score is above a threshold and its remaining privacy budget is sufficient.

Possible score:

> $\text{ResourceScore} = \alpha \times \text{Battery} + \beta \times \text{Bandwidth} + \gamma \times \text{MemoryAvailability} + \delta \times \text{DataContribution} - \lambda \times \text{PrivacyCost}$

Phase 3: Add precision-aware DP

Test:

1. floating-point DP noise,
2. quantized/fixed-point DP noise,
3. rounded Gaussian/Laplace noise,
4. discrete noise approximation.

Then measure:

- difference between intended σ and implemented σ,
- effect on ε estimation,
- model accuracy,
- convergence,
- memory/energy cost.

Phase 4: Add communication-aware DP

test:

1. DP without compression,
2. DP + gradient clipping,
3. DP + sparsification,
4. DP + quantization,
5. DP + sparsification + resource-aware participation.

clipping-threshold adaptation + sparsified/noised updates.

The main question:

> Does communication reduction amplify or reduce DP noise impact?

This is useful because sparsification may reduce communication, but it can also distort noisy updates and affect convergence.

Phase 5: Optional future extension

Method	Future role
Split FL	Reduce local training memory; apply DP to smashed activations
TEE / TrustZone-M	Protect noise seeds, keys, and local DP routine
Secure Aggregation	Combine with DP for server-side update privacy
ZKP	Verify that clients followed DP/noise protocol
HE / blockchain	Not MCU-core; gateway/server-side support only

Resource-aware privacy adaptation methods in recent FL-IoT literature

Method	Paper	Problem solved and how	remaining gap
Personalized Local Differential Privacy (personalized LDP)	ACSFL: Adaptive client selection-based FL with personalized differential privacy for heterogeneous AIoT environments, 2025	different AIoT clients have different privacy needs and resource conditions. ACSFL combines adaptive client selection with personalized local DP, using node heterogeneity, privacy budget, and contribution to select clients.	does not fully address TinyML-class memory limits
Dynamic privacy-budget allocation	**Differential Privacy Federated Learning Based on Adaptive Adjustment**, Cheng et al., 2025	Solves the privacy–accuracy trade-off caused by fixed DP noise. The method dynamically adjusts privacy budget according to client data volume and training requirements, reducing unnecessary noise and privacy-budget loss.	Good for adaptive DP, but not specifically designed for low-memory TinyML or unstable IoT communication. Resource factors like battery, CPU, RAM, and communication mode are not deeply integrated.
Adaptive modular privacy-preserving learning	Adaptive resource-aware and privacy-preserving federated edge learning framework for real-time Internet of Medical Things applications	it combines **resource-aware local training**, **privacy-preserving computation**, and **edge/IoMT constraints**.	Very relevant to resource-aware privacy, but it is focused on IoMT healthcare and edge/fog-style systems.
Adaptive DP + homomorphic encryption + reinforcement-learning thresholding	**SecuFL-IoT: An adaptive privacy-preserving federated learning framework for anomaly detection in smart industrial networks**, Scientific Reports, 2026	Solves privacy leakage, poisoning, and communication inefficiency in IIoT anomaly detection. It integrates DP, lattice-based homomorphic encryption, and reinforcement-learning-based threshold adjustment	it is IIoT anomaly-detection-specific. It also introduces encryption latency and assumes a static network topology, which limits adaptability in highly dynamic IoT networks.

Lightweight DP for edge/IoT analytics	**Edge computing and federated learning for privacy-preserving IoT analytics**, 2025	Solves the issue that privacy protection should not destroy accuracy or efficiency. It uses adaptive learning rates and a lightweight DP mechanism for IoT analytics	It does not fully answer how much noise/encryption each low-resource client should use.
Selective Homomorphic Encryption (SHE)	**Tariq et al., AI-Driven Lightweight and Trustworthy FL Framework for Resource-Constraine d Edge-IoT**, IEEE TCE, 2025	Solves the high cost of full HE by encrypting only sensitive/critical parameters. The paper combines adaptive pruning, quantization, communication-frequency optimization, and Selective Homomorphic Encryption for resource-constrained Edge-IoT	still complex. SHE adds key-management overhead and may provide partial privacy rather than full-update protection.
Hybrid encryption: partial homomorphic encryption + DP	**SAFEL-IoT: Secure Adaptive Federated Learning with Explainability for Anomaly Detection in 6G-Enabled Smart Industry 5.0**, Electronics, 2025	Solves secure update transmission under resource-constrained IIoT. It uses dynamic aggregation, partial homomorphic encryption, DP, and SHAP explainability.	Strong for hybrid privacy, but it assumes advanced 6G/network-slicing infrastructure. It may be too complex for simple TinyML + WiFi/LoRa/GSM experiments.

What has literature proposed to reduce HE / blockchain / SMPC overhead

Area	Existing Methods to reduce Cost	Remaining Limitation	plan?
Homomorphic Encryption (HE)	Selective HE, partial encryption, lightweight HE, encrypt only important gradients	Still high computation, ciphertext expansion, key management overhead	
SMPC / Secure Aggregation	Group-based secure aggregation, clustered FL, secret sharing only within small groups	High communication cost, dropout handling, collusion risk	
Blockchain	Permissioned blockchain, hash-only logging, off-chain model storage, lightweight consensus, reputation systems	Ledger synchronization, storage, and consensus are heavy for MCUs	

Papers:

1. A Selective Homomorphic Encryption Approach for Faster Privacy-Preserving Federated Learning - 2025
2. FedSHE–CQ: A Communication-Efficient Homomorphic Encryption Framework for Federated Learning -2025
3. MaskCrypt: Federated Learning with Selective Homomorphic Encryption - 2025
4. Group Verifiable Secure Aggregate Federated Learning Based on Secret Sharing -2025

four layers for the framework:

Layer 1: Device profiler
Measures or estimates:

- available SRAM
- Flash budget
- battery level
- CPU cycles
- network bandwidth
- model size
- update size
- privacy requirement

Layer 2: Privacy-cost estimator
Estimates the expected cost of each method:

- Local DP cost
- secure aggregation cost
- SHE/PHE cost
- TEE cost
- split FL cost

Layer 3: Adaptive privacy selector
privacy mode options:

- Mode 0: No PPM baseline
- Mode 1: Lightweight Local DP
- Mode 2: Local DP + TEE-protected seed/key
- Mode 3: Local DP + lightweight secure aggregation
- Mode 4: Split FL + Local DP
- Mode 5: SHE only for sensitive layers/gradients
- Heavy methods like full HE/blockchain/ZKP are kept as comparison or gateway-side methods, not direct MCU methods

Layer 4: Evaluation engine
Measures:

- accuracy
- training time
- communication overhead
- RAM/Flash use
- energy per FL round
- privacy leakage risk
- battery lifetime impact

Build in this order (each is a self-contained cell block):

1. MFCC data preprocessing + Dirichlet non-IID split across 50 clients
2. TinyCNN model (3-layer INT8-quantized CNN)
3. Device profiler (mock energy/memory/bandwidth values)
4. Dual-mode decision logic
5. Local training loop (quantized SGD, 3 epochs)
6. Cosine similarity filter + INT8 compression
7. FedAvg aggregation
8. Last-layer personalization
9. Evaluation loop (50 rounds, plot accuracy/energy/comm cost)

See subtabs

Possible Baseline Approach

Baseline Layer 0: Centralized TinyML Model

Before FL, train the same lightweight model in a normal centralized way.

Purpose:

To know the best possible performance of the model before splitting data across clients.

We will use a public dataset and train the tinyCNN or MLP model to see its performance without FL integration.

Baseline Layer 1: Standard FL without Privacy

In this setup, there is no DP, no compression, no selective participation, no encryption, and no adaptive mechanism.

Basic setup

Item	Suggested Setup
FL algorithm	FedAvg
Clients	10 or 20
Client participation	100%
Local epochs	1–5
Rounds	50–100
Dataset split	IID and Non-IID
Model	Lightweight TinyML-compatible model
DP	No
Compression	No
Resource adaptation	No

Baseline Layer 2: Fixed-DP FL

add a simple fixed Differential Privacy mechanism. Every client uses the same clipping norm and the same noise multiplier.

We will use the exact same dataset, model, clients, rounds, and local epochs as Baseline 0. The only change is:
Client update → clipping → fixed DP noise → server aggregation

DP Item	Example
Clipping norm	C = 1.0
Noise multiplier	σ = 0.5 / 1.0 / 1.5
Delta	1 / number of training samples
Epsilon	calculated using privacy accountant
DP level	client update-level or gradient-level

Expected output:

Version	Accuracy	Accuracy Drop	Epsilon	Noise	Rounds to Converge	Training Time
Standard FL	92%	—	∞	0	45	—
Fixed-DP FL	87%	-5%	4.2	1.0	60	

Baseline Layer 3: Communication and Resource Logging

For communication, log:

Metric	Meaning
Upload per client	Size of client update sent to server
Download per client	Size of global model sent to client
Communication per round	Total upload + download
Total communication	Overall communication after all rounds
Rounds to target accuracy	Communication efficiency indicator

For resources, log:

Metric	Meaning
Local training time	Time needed by each client
Aggregation time	Server-side aggregation cost
Round time	Total time per FL round
Model size	TinyML feasibility
Update size	Communication feasibility
RAM/CPU usage	Resource feasibility if measurable
Inference time	TinyML deployment feasibility

Output 1: Dataset and Client Split Report

This should show:

Client	Samples	Class Distribution	Resource Type
Client 1	500	Class 0: 200, Class 1: 300	Low
Client 2	700	Class 0: 100, Class 1: 600	Medium
Client 3	1000	Class 0: 500, Class 1: 500	High

This proves that the dataset was properly split across clients.

Output 2: Standard FL Result

This should show:

Metric	Value
Final accuracy	—
Final loss	—
Best accuracy	—
Rounds to convergence	—
Total training time	—
Total communication	—

This proves that the FL pipeline works.

Output 3: Fixed-DP FL Result

This should show:

Metric	Standard FL	Fixed-DP FL
Accuracy	—	—

Loss	—	—
Rounds to convergence	—	—
Training time	—	—
Communication	—	—
Epsilon	∞	—

This proves that privacy has been added and its cost is measurable.

Output 4: Curves/Figures

Minimum figures:

1. Centralized vs FL accuracy.
2. FL accuracy vs rounds.
3. FL loss vs rounds.
4. Standard FL vs Fixed-DP FL accuracy.
5. Communication cost per round.
6. Training time per round.
7. IID vs Non-IID comparison.

These figures will help the supervisors quickly see baseline progress

To track baseline completion:

Stage	Evidence
Dataset ready	Client-wise dataset split saved
Model ready	Lightweight model trains centrally
FL ready	FedAvg runs across clients
Logging ready	Accuracy, loss, time, communication saved
DP ready	Fixed clipping + fixed noise added
Privacy report ready	Epsilon and accuracy drop reported
Comparison ready	Standard FL vs Fixed-DP FL table produced
Reproducibility ready	Same config can be rerun with same seed

Technical evaluation of different **Federated Learning (FL) simulators**

Our goal is a pipeline where you plug in *any* real ML model or optimization strategy and automatically get back measured CPU, memory, and power usage per client

To build this framework, we have to combine a **measured calibration rig** with an **abstract simulator**.
[our Strategy / Model] │
▼
│ **STAGE 1:** │ **Calibration** ││ —► Runs a brief 1-round test on a real CPU/GPU, Tracks actual memory overhead and CPU wall-clock
▼ **(Extracts: FLOPs, Memory Footprint, Message Bytes)**
│ **STAGE 2:** │ **Falafels Sim** │ │ ──► Scales up to thousands of heterogeneous virtual clients, computes macro-level execution time and energy burn

Stage 1: The Calibration Profile (Using BouquetFL or a Profiler)

Before running large-scale simulations, run a single-round physical micro-benchmark of our model and chosen optimization strategy on a real CPU. This anchors the abstract simulator to empirical metrics.

- **Purpose:** To discover the raw physical baseline (FLOPs, network packet bytes, and peak RAM footprint) required by a single client before scaling up.
- **Mechanism:** Execute one local training round using raw Flower or a standard PyTorch script on a local machine. Use execution profilers (such as CodeCarbon, perf, or RAPL) to catch the baseline wall-clock time and memory consumption.
- **Extracted Parameters:** * Total Local Compute Workload (X FLOPs).
    - Model Upload/Download size (M bytes).
    - Peak Memory Allocation (R kilobytes).

Stage 2: The Large-Scale Simulator (Using Falafels)

Once we have those baseline metrics, you feed them into Falafels. Inside Falafels, define our diverse client network using a SimGrid XML file. We can set trainer_1 to look like a weak microcontroller (low clock speed, low wattage) and trainer_2 to look like a high-end edge device. Falafels will then instantly simulate how that strategy scales across hundreds of distinct, resource-budgeted virtual clients.

Layer 1:

Port the parameters extracted from Layer 0 into Falafels, a discrete-event simulator built on SimGrid that runs efficiently on a single CPU host.

- **Purpose:** To scale our experiment instantly up to dozens or hundreds of virtual edge clients without needing actual cluster hardware.
- **Mechanism:** Define our client network using a SimGrid Platform XML file. Each virtual client is assigned unique hardware properties representing real-world hardware limits:
    - Compute Capacity (speed in flops/s).
    - Hardware Cores (core count).
    - Power Consumption States (wattage_per_state defining Idle, OneCore, and AllCores power draw).

Layer 2:

To test different optimization strategies, explicitly alter the workload profiles of the virtual clients according to the specific mathematical behavior of the algorithms:

- **FedAvg:** Uses the baseline compute work (X FLOPs) and standard network payload (M bytes).
- **FedProx:** Injects extra compute FLOPs per step to model the overhead of calculating L_2 proximal term. Payload remains M bytes.
- **SCAF FOLD:** Models an extra compute factor for control variates and increases the uplink/downlink payload sizes to 2M bytes to simulate the transmission of server/client control variates

What the Pipeline Outputs Will Be

1. CPU Metrics (Modeled Time)

- The Output: Simulated execution time/round duration per client (in virtual seconds).
- How it's calculated: Falafels takes the FLOP count you extracted from Stage 1 and divides it by the virtual client's assigned CPU speed (flops/s) defined in your XML. It automatically factors in delays if multiple virtual clients are sharing the same host hardware.

2. Power Consumption Metrics (Natively Modeled)

- The Output: Total energy consumed per client (expressed in Joules).
- How it's calculated: Falafels leverages SimGrid's energy plugin. It looks at the client's virtual power states (e.g., drawing 1.8W when idle vs. 5.1W when all CPU cores are active) and multiplies that wattage by the simulated CPU time.

  Example:

Client ID	Selected Strategy	Simulated CPU Round Time	Total Energy Consumed	Peak Memory Profile	Memory Budget Feasibility
Client 1	FedAvg	12.5 seconds	33.7 Joules	180 KB	**PASS** (Within 256KB ceiling)
Client 1	SCAFFOLD	28.1 seconds	75.8 Joules	360 KB	**FAIL (Out of Memory)**
Client 2	FedProx	4.2 seconds	21.4 Joules	12.4 MB	**PASS** (Within 512MB ceiling)

3. Memory Usage Metrics (Statically Tracked)

- The Output: A pass/fail feasibility report per client (e.g., *"Client 3 requires 512KB RAM; Hardware limit is 256KB -> Out of Memory"*).
- The Catch: Falafels does not natively track RAM or simulate OOM crashes. To get this output, your pipeline must inject a static memory calculation layer. It takes the peak memory measured in your Stage 1 calibration run and compares it mathematically against the hard memory ceiling you assigned to that client.

Example:

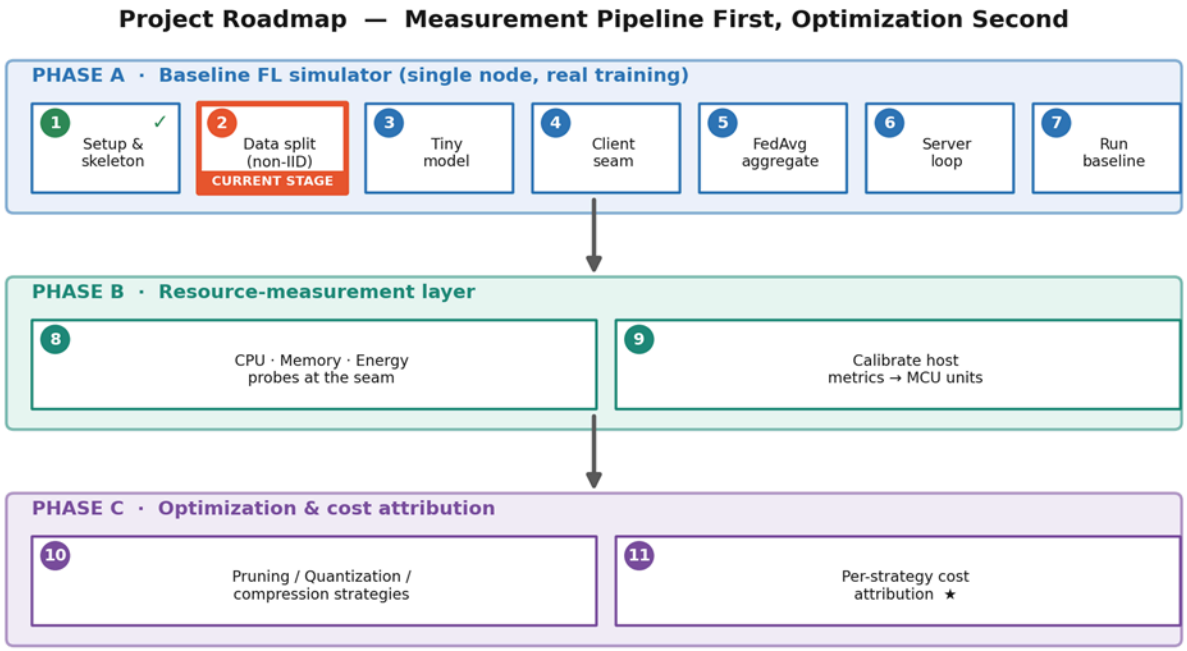
Client ID	Selected Strategy	Simulated CPU Round Time	Total Energy Consumed	Peak Memory Profile	Memory Budget Feasibility

Client 1	FedAvg	12.5 seconds	33.7 Joules	180 KB	**PASS** (Within 256KB ceiling)
Client 1	SCAFFOLD	28.1 seconds	75.8 Joules	360 KB	**FAIL (Out of Memory)**
Client 2	FedProx	4.2 seconds	21.4 Joules	12.4 MB	**PASS** (Within 512MB ceiling)

Federated Learning Resource-Measurement Pipeline for MCU-Class Clients

Project Plan and Current Status

1. Project Roadmap



The work runs in three phases. **Phase A** builds a working baseline that runs real training on simulated non-IID clients on a single node. **Phase B** adds the resource-measurement layer — CPU, memory, and energy probes — and calibrates host-side numbers into MCU-scale units. **Phase C** introduces the optimization strategies and uses the measurement layer to attribute their costs.

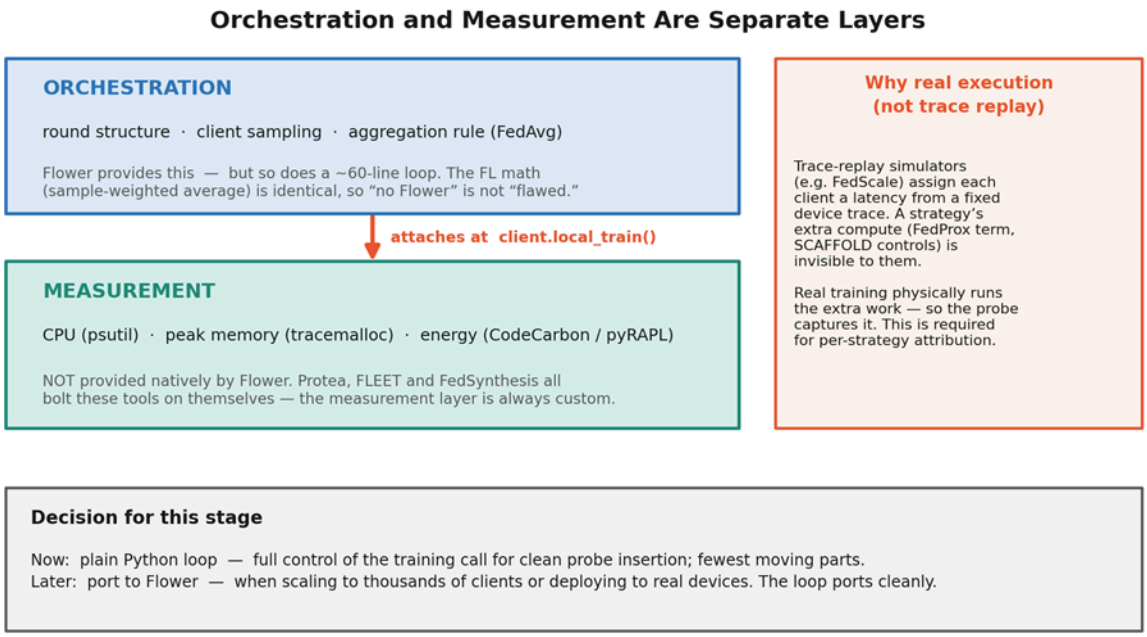
Current position: Phase A, step 2 — the non-IID data partition. Project scaffolding and environment (step 1) are complete; the partition is now being validated before the model is built.

2. Baseline Simulator Architecture

The baseline runs entirely on one host; each client is a software object, not a device, and training is CPU-only to keep timing honest (MCUs have no GPU). A shared data pool is split across clients with a Dirichlet label skew ($\alpha = 0.1$), so each client has a few dominant classes — the very non-IID condition. In each round, the server samples a subset of clients; each selected client runs real PyTorch SGD on its private shard inside a single method, local_train(); the resulting weight updates are combined by sample-weighted FedAvg; the global model is updated and evaluated.

The local_train() call is the deliberate **measurement seam**. This is where Phase B's probes attach, because it is exactly where a client's **compute, memory, and energy** are consumed — and where different strategies do different amounts of work.

3. Measurement Approach



Two responsibilities are kept separate by design: **orchestration** (how rounds are structured, who trains, how updates combine) and **measurement** (what each client's training costs). Orchestration is small and standard. Measurement is always custom — no federated learning framework reports CPU, memory, and energy natively; published systems add tools such as psutil and CodeCarbon themselves, at the training call.

4. Why Flower Is Not Used at This Stage

Flower is a federated-learning framework and a likely future component of this project. It is set aside at this stage for:

- **It would not remove Phase B's work.** Flower provides orchestration but does not measure resources, so the measurement layer would still have to be built by hand.
- **It complicates the seam.** Flower runs client training behind a distributed-execution layer (Ray actors and strategy callbacks), which makes deterministic, fine-grained probe insertion harder than in a plain loop where training is an ordinary method call.
- **It is not needed for correctness.** The federated-learning mathematics is identical either way — sample-weighted averaging — so building without Flower does not make the baseline less correct.

For a measurement-first prototype on a single node, a plain Python loop gives full control of the seam with the fewest moving parts. The plan is to port to Flower **later**, once the measurement layer is validated and the need is for scale (thousands of clients) or deployment to real devices.

5. Current Status and Next Steps

Completed: project structure and environment; the client / server / aggregation separation built around the measurement seam.

In progress: the non-IID data partition, currently validated by confirming the per-client class skew at $\alpha = 0.3$ (each client dominated by 2–3 classes).

Immediate next steps, in order:

1. Confirm the partition skew is correct (the gate before any model work).
2. Build the tiny CNN model and report its parameter byte-footprint.
3. Implement the client local_train() seam.
4. Implement sample-weighted FedAvg aggregation.
5. Implement the server round loop.
6. Produce a first converging baseline run on the synthetic data, then swap in CIFAR-10.

The resource-measurement layer (Phase B) begins **only** once this baseline converges.

6. Evidence Base

The design choices above are grounded in current literature and tooling, verified against source repositories and issue trackers:

- **No framework measures resources natively.** Profilers are added on top: Protea (psutil-based client profiling for Flower), FLEET (psutil integrated into Flower for resource-aware selection), and FedSynthesis (CodeCarbon integrated into Flower for energy/carbon tracking).
- **Orchestrator plus a separate energy model.** FedZero (ACM e-Energy 2024) pairs Flower with the Vessim energy-system simulator — the same "harness + separate energy model" decomposition this plan mirrors. Its code is public.
- **Trace-replay is strategy-blind.** FedScale (ICML 2022) drives client latency from real device traces — heterogeneous per device but not per strategy, which is why real execution is used here for attribution.
- **Dataset.** CIFAR-10 (uoft-cs/cifar10 on Hugging Face; 32×32 colour, 10 classes) is one of the MLPerf Tiny benchmark tasks — a reasonable stand-in while the pipeline is built. The audio keyword-spotting setting from the reference paper is a later one-function swap behind the same data contract.

client num = 10, client per round = 10, num rounds = 20, dirichlet alpha = 0.3, seed = 42, local epoch = 20, mcu batch size = 1, lr= 0.01, target precision bytes = 4, see results.

Measurement Axis Validation

mcu_class_fl_sim — resource instrumentation validated against peer-reviewed literature

Scope note: literature groundings below are single-pass. The adversarial refutation stage did not complete, so entries marked "quote not retrieved" have confirmed venues but unverified claims. Findings in Table 3 were produced by executing the code and are independent of that caveat.

1. Axes with literature grounding

Axis	Grounding paper	Base algorithm in FL / on-device-training literature	Verification status
A1 Compute FLOPs & MACs, fwd + bwd	Hoffmann et al., Chinchilla, NeurIPS 2022 — App. F: backward = 2x forward Sze et al., Proc. IEEE 2017 — MAC = 2 ops	Report MACs, forward only, whole-net (MCUNet, MCUNetV3). MCUNetV3 does not split fwd/bwd — your split is finer than prior art.	Venue confirmed; quotes not retrieved
A2 Memory peak training RAM	Lin et al., MCUNetV3, NeurIPS 2022 Rajbhandari et al., ZeRO, SC20 — params/grads/optimizer decomposition	Analytic, implementation-agnostic tensor census, cross-checked by on-device profiling. MCUNetV3 measures memory at batch=1 on the MCU while simulating accuracy on GPU — the same L3/L4 decoupling you use.	VERIFIED verbatim from NeurIPS proceedings PDF
A3 Storage / Flash	None found (see Table 2)	MCUNet reports Flash profiled on-device with TinyEngine; no published work counts weight-persist events.	Ungrounded
A4 Communication uplink / downlink bytes	McMahan et al., FedAvg, AISTATS 2017 Lai et al., FedScale, ICML 2022	McMahan measures ROUNDS, not bytes. Byte-level accounting is a FedScale-era systems metric, not the FedAvg convention.	McMahan confirmed rounds-based; FedScale extraction incomplete
A5 Time modeled MCU latency	Zhang et al., nn-Meter, MobiSys 2021 — Best Paper Banbury et al., MLPerf Tiny, NeurIPS 2021 D&B	Kernel-level latency predictors or direct on-device measurement. nn-Meter reports 89.2% +/-10% accuracy vs. markedly lower for FLOPs and FLOPs+MAC predictors — a single-scalar MAC-rate model is the documented worst-performing baseline.	VERIFIED — 89.2% figure extracted from nn-Meter PDF
A6 Energy (host) RAPL via pyJoules	Khan et al., RAPL in Action, ACM TOMPECS 3(2), 2018	RAPL with explicit domain selection (never sum all domains), sampling intervals above ~50 ms, idle baseline subtracted. Reported MAPE 3.1% (Haswell) / 4.0% (SandyBridge).	VERIFIED
A6 Energy (modeled) MCU energy equation	Qiu et al., Carbon Footprint of FL, JMLR 24 Patil et al., POET, ICML 2022 Horowitz, ISSCC 2014 — per-op energy	E_tx = E_elec * k + eps * k * d^2. Your per-byte term corresponds to E_elec; the distance/amplifier term is omitted. POET uses measured per-op energy on real MCUs rather than a closed-form model.	Venues confirmed; extraction incomplete

	Heinzelman et al., radio model (HICSS 2000 / IEEE TWC 2002)		
L4 Calibration device constants	Banbury et al., MLPerf Tiny, NeurIPS 2021 D&B MCUNet model zoo (TinyEngine, STM32F746) Llisterri Gimenez et al., Electronics 11(4), 2022	Measure on hardware. MLPerf Tiny supplies (MACs, measured Cortex-M4 latency) pairs: DS-CNN 2.66M, MobileNetV1 7.49M, ResNet-8 12.5M, FC-AE 0.26M MACs.	MAC counts extracted; latency pairing pending

2. Axis without grounding

Axis	Search outcome	Nearest supporting literature	Required action
A3 Flash write endurance (weight_persist_events)	No FL or TinyML paper counts weight-persist events. Two independent search passes. The metric appears genuinely novel.	Intermittent computing (Mementos, Hibernus) explicitly models NVM write cost; Coffee filesystem (sensor networks); TFLite Micro, MLSys 2021 for runtime binary size.	Argue relevance from first principles, not by citation. Add pe_cycle_budget to DeviceProfile; derive device lifetime = budget / (rounds x writes per round). Currently the field is logged but never consumed by estimate.py.

3. Defects blocking certification

Each item below was reproduced by running the instrumentation, not inferred from reading it. Axes A1, A2 and A6 cannot be certified until items 1-4 are resolved.

#	Defect (found by executing your code)	Location	Impact on validity
1	flops_per_layer["per_sample_backward"] is always empty. Forward runs outside the backward counter's ModuleTracker context, so backward ops get no module attribution.	instrumentation/compute.py:47	Silent failure. Per-layer backward data does not exist despite being logged.
2	per_sample_forward double-counts ~2x. get_flop_counts() returns a hierarchy; both TinyCNN.features and TinyCNN.features.0/3/6 are kept.	instrumentation/compute.py:47	Summing the per-layer table gives 2x the true 5,604,608 FLOPs. The scalar flops_forward is correct — only the dict is wrong.
3	Activation census over-counts 78%. Repo counts 67,658 elements; true PyTorch saved-tensors ~37,963 floats. Conv output and ReLU output counted as distinct.	instrumentation/memory.py:55	peak_ram_bytes inflated. Also omits activation gradients (dL/da) and never checks requires_grad — breaks under sparse update.
4	RAPL domains summed without deduplication. Core and uncore are	instrumentation/host_energy.py:41	2-3x energy inflation when psys is present. Invalidates all host energy numbers.

	subsets of package; psys overlaps package+dram.		
5	wall_time_s is not stable on this host. Your own logs: round 3 client 1 = 557 s vs round 1 = 100 s for identical work; cpu/wall ratio fell 6.0 to 1.6.	clients/client.py:45-57	Thermal throttling and host load dominate. cpu_time_s is worse — it sums OpenMP threads including busy-wait spin (ratio 9.7 observed = thread count).
6	Flash budget omits the local dataset. CIFAR-10 at 500 samples x 3 KB = 1.5 MB, already over the 1 MB budget before any weights.	resource_models/estimate.py	May invalidate the headline viability conclusion outright.
7	Cross-axis precision contradiction. model_flash_bytes respects target_precision_bytes (INT8 -> 24 KB) but uplink/downlink stay fp32 at 99,917 B in the same record.	common/serialization.py + storage.py	Two contradictory precision assumptions inside one JSONL row.
8	Silent zero-defaults: g = lambda k: rec.get(k) or 0.	resource_models/estimate.py:14	Missing peak_ram_bytes becomes 0, so fits_in_sram returns True. Silent false positive on your central claim.
9	torch.save framing charged to the radio: 99,917 B vs 96,936 B payload = 3.08%, and it is per-tensor (~370 B each), not proportional to model size.	server/server.py:46-47	Non-scale-invariant: depthwise-heavy models penalised disproportionately. state_num_bytes exists for this and is dead code.
10	verify_monitors.py has no Axis-4 assertion.	experiments/verify_monitors.py	Communication is the only axis with no hand-computed ground-truth test, breaking your own verification discipline.

4. Verification against datasheets

The two compute constants are algebraically coupled: P_active = joules_per_mac x effective_macs_per_cycle x clock_hz. Recalibrating one without the other silently changes the implied device power, so D1, D2 and D10 must be resolved together.

#	What to verify	Against	Why it matters
D1	STM32F746 active power at 216 MHz. Your constants imply P_active = J_mac x macs_per_cycle x clock = 0.33 W.	ST STM32F746 datasheet, IDD run-mode tables	Confirms joules_per_mac and effective_macs_per_cycle are jointly consistent. Preliminary check found ~0.36 W (peripherals off) — needs a formal citation.
D2	nRF52840 active power at 64 MHz. Your constants imply ~16 mW.	Nordic nRF52840 Product Specification, CPU run current	Same coupling check. Preliminary range 10-19 mW.
D3	nRF52840 radio TX current at 0 dBm. Your constants imply ~15 mW radio power.	Nordic nRF52840 PS, radio current consumption	Validates joules_per_byte_tx. Preliminary: 4.8-5.3 mA at 3 V ~= 15 mW — consistent.

D4	STM32F746 SRAM region layout. 320 KB is NOT contiguous (SRAM1 / SRAM2 / DTCM).	ST reference manual, memory map	fits_in_sram compares against one flat 320 KB pool. A tensor cannot span regions, so the check is optimistic.
D5	nRF52840 SoftDevice / BLE stack RAM reservation.	Nordic SoftDevice S140 specification	Tens of KB must be subtracted from the 256 KB before fits_in_sram is meaningful.
D6	Flash P/E cycle endurance for both parts.	ST and Nordic datasheets, endurance tables	Without this, weight_persist_events cannot be converted into a device lifetime and the metric stays inert.
D7	Flash page / sector erase granularity.	ST and Nordic reference manuals	Persisting weights requires erase-before-write of a whole sector, so one logical write may cost far more than model_flash_bytes.
D8	Flash write energy and write time.	Datasheet programming current and timing specs	Currently absent from both the energy and latency models. Preliminary: nRF52840 ~44 mA during write.
D9	Sustained BLE application-layer throughput.	Nordic throughput application notes; BLE 5 spec	radio_bps = 1e6 is the PHY rate, not the app rate. Real sustained throughput is roughly 30% of PHY, so t_comm and comm energy are optimistic by ~3x.
D10	Cortex-M7 and M4F effective MACs/cycle for fp32 and INT8 with CMSIS-NN.	ARM CMSIS-NN benchmarks; Cortex-M7 TRM (dual-issue, VFMA)	Validates the 0.2 / 0.15 placeholders — the single largest source of error in mcu_latency_est_s.

5. Remaining literature and empirical verification

#	Task	Notes
L1	Retrieve the Chinchilla App. F quote stating backward = 2x forward.	Load-bearing for the A1 fwd/bwd convention. Venue confirmed, quote not yet extracted.
L2	Confirm FedScale reports byte-level communication and how it computes bytes.	Determines whether len(payload) or pure tensor bytes is the expected convention.
L3	Extract the Heinzelman E_elec term and confirm your per-byte model is that term without the amplifier component.	Grounds the modeled energy equation.

L4	Find a published TFLite Micro / TinyEngine binary size to justify runtime_flash_bytes = 50 KB.	TFLite Micro, MLSys 2021 is the likely source. Currently an unsourced constant.
L5	Confirm MCUNetV3 optimizer-state convention (it states Adam = 3x SGD) against your opt_mult = 2.	Possible off-by-one in the memory multiplier. ZeRO uses K = 12 for mixed-precision Adam.
E1	Calibrate effective_macs_per_cycle from MLPerf Tiny (MACs, measured latency) pairs.	Peer-reviewed calibration source, already identified. Note these are inference numbers — state the caveat when reusing for training.
E2	Re-run all timing and energy experiments on an isolated, thermally stable Linux host.	Current wall_time_s and cpu_time_s are unusable. RAPL also needs Linux.
E3	Check pyJoules 32-bit RAPL counter wraparound (~60 s at high power) against observed round times of 100-557 s.	Live risk of silent counter overflow in your longest rounds.
E4	Add an Axis-4 hand-computed assertion to verify_monitors.py.	Closes the only gap in your ground-truth test coverage.
E5	Re-derive peak_ram_bytes from PyTorch saved-tensor semantics rather than all leaf outputs.	Fixes the 78% over-count and aligns with MCUNetV3 analytic profiling.